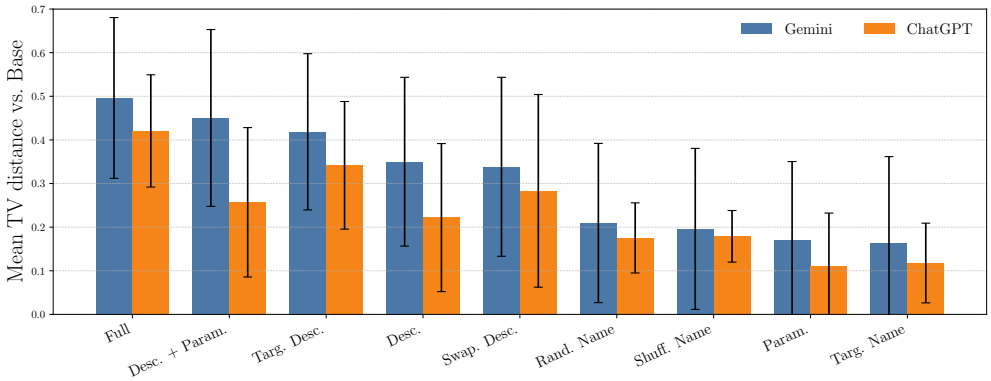

Figure 5: Mean total-variation (TV) distance from the base selection distribution (no perturbation) to the distribution pertaining to each metadata perturbation (higher = larger shift). Blue bars show results for Gemini and orange bars for ChatGPT. The error bars denote standard deviation across clusters. The run-to-run standard deviation is left out; max run-to-run variability of per-run mean TV was 0.084.

training objective can alter these tendencies, but overall alignment underscores that tool-selection bias patterns are not isolated quirks but mostly reproducible phenomena across LLMs.

Ablation and Sensitivity Analysis. We also analyze the robustness of tool-selection bias across several factors (full results in Appendix E). *Temperature:* raising temperature reduces bias modestly by softening extreme preferences. *Top-p:* increasing top-p has only a negligible effect. *Model size:* larger models exhibit noticeably less bias than smaller ones. *API ordering:* cyclic vs. random permutations produce very similar outcomes, indicating intrinsic preferences dominate. *System prompts:* rewording or restructuring prompts shifts which tools are favored, but does not remove bias.

4.3 Are tool-selections driven by human-interpretable heuristics?

We investigate the drivers of bias along three complementary axes: (i) feature-level correlations between API attributes and selection rates, (ii) direct interventions on API metadata, and (iii) biased continued pre-training (CPT) to test whether exposure alone can plant preferences. We summarise our results below, with full details in Appendix F.

Which API-level features predict selection rates? The results show a consistent pattern: (1) Semantic similarity between queries and API / tool descriptions is the strongest predictor of selection (Table 4). By contrast, structural or stylistic attributes (e.g., parameter count, promotional wording) exhibit little consistent influence. (2) Linear regression reveals that surface-level semantic alignment is the primary signal but leaves a lot unexplained ($R^2 < 0.4$ as can be seen in Figure 13), and (3) Random forests fail to offer meaningful improvement.

How do metadata interventions affect API selection? Figure 5 shows the TV distance from the base selection distribution to the distribution obtained after a certain metadata perturbation, averaged over clusters and experimental runs. It shows a clear hierarchy: logically minimizing semantic signal by scrambling the description, name, and parameters causes the biggest shift. For Gemini, corrupting descriptions+parameters yields the second largest, most reliable shifts in selection (0.450 ± 0.203 TV), followed by scrambling the description of the most popular tool (0.419 ± 0.179), and even description swaps meaningfully steer choices (0.338 ± 0.205). By contrast, name-only perturbations are smaller and have higher variance, and parameters-only are the least impactful. We see similar results for ChatGPT; description perturbation is more impactful than name/parameter perturbations. Overall sensitivity is higher for Gemini than ChatGPT (mean TV 0.310 vs 0.234), indicating greater responsiveness to metadata changes.

We show similar results across clusters in Figure 14; however, we observe that manipulating the description of certain tools has mixed effects from cluster to cluster. It tells us that the impact of

description tampering is context-dependent: the same intervention can invert, redistribute, or barely change preferences, underscoring that tool choice emerges from multiple interacting cues.

Together, these patterns indicate that description-level semantics are the primary cues models use to discriminate among functionally similar APIs. Name perturbation alone tends to inject noise without consistent effects. Finally, bias persists under minimal semantic signal (only leaving certain parameter schema fields intact), implying selection behavior sometimes relies on residual, non-obvious priors rather than solely on coherent, human-interpretable heuristics. For an analysis of the impact of metadata perturbations on bias, see Appendix H.

Does additional pre-training exposure to one endpoint change selection distribution? We run biased continued pre-training on Qwen3-8B in which the training corpus is saturated with metadata for a single target endpoint (Text Language by API-Ninjas)[2]. All inference settings, prompts, and decoding parameters are held fixed; only the checkpoint changes (see full training and inference hyperparameters in Appendix I). We then compare selection rates within the same Language Identification cluster before CPT and after $\sim$1/3, $\sim$2/3, and one full epoch of biased training (See Figure 17 for full figure). We observe that biased CPT substantially increases the selection of the exposed endpoint but does not fully determine choice. The target endpoint's share rises from 0.006 (base) to 0.122 after 1/3 epoch, remains 0.122 at 2/3 epoch, and nudges to 0.128 at one epoch. This is an absolute gain of $\sim$12 percentage points (over 20$\times$ relative). Most of the shift is realized early, suggesting a quickly saturating response. This demonstrates that biased exposure during training can directly shape tool-selection preferences in favor of the exposed endpoint. However, since the target never approaches a dominant share, pre-training exposure explains only part of the bias, and additional factors still shape tool choice.

4.4 Can we mitigate the observed bias?

After demonstrating the existence and causes of bias, we seek to mitigate it. Our approach is simple: models often recognize which APIs can solve a certain task, but still exhibit skewed preferences among interchangeable endpoints. Hence, we propose to decouple recognition from selection via a lightweight debiasing module. This module uses a smaller LLM (Qwen3-14B), prompted to return only the subset of APIs from the candidate list that can solve the given query. We then choose uniformly at random from this subset, ensuring that each valid API has the same expected probability of selection. This eliminates positional or metadata-based favoritism while maintaining task coverage. See Appendix D for benchmark details and evaluation metrics. This section showcases the potential of the method and outlines how it can help.

Subset selection avoids spurious inclusions of incorrect APIs while maintaining high coverage of correct APIs, thereby mitigating bias without sacrificing performance. Table 2 summarizes results for Qwen3-14B as the subset selector. Overall micro-precision is $\sim$1.00 (0.9964), meaning the selector almost never adds tools to the candidate set that cannot solve the user's task. This is a desirable property as it means that our mitigation module is unlikely to output an incorrect tool, and therefore, performance will not suffer. Note that since all ground-truth set size classes have nearly the same number of queries, micro- and macro-precision are effectively equivalent; we report micro-precision for simplicity. Micro-recall is $\sim$0.89 (0.8856), so on average the selector itself is not biased and retains most ground-truth tools, with an exact set match of 0.69 across all instances. Broken out by the size of the ground-truth set K, precision remains essentially perfect across the board, while recall varies: it is strongest at K=4 (0.9633) and somewhat lower at K=2 (0.7717) and K=5 (0.8610). The corresponding exact-match rates reflect the same pattern (notably 0.9100 at K=4). In practice, this means the subset filter very rarely introduces distracting tools (good for performance), but it can occasionally omit a true option when K is small or large. Taken together, these results suggest subset selection is a promising first line of defense: it does not lead to spurious inclusions (high precision) and maintains high coverage of correct APIs on average, and thus, less bias downstream (strong recall).

Our mitigation method successfully alleviates tool-selection bias. Figure 6 shows the empirical selection distributions of ChatGPT 4.1-mini before and after applying our mitigation method over

[2]We use this model because its smaller size makes training tractable on modest hardware.

Table 2: Subset–selection performance for Qwen3 (14B) (overall and by ground–truth set size K).

Overall	
Micro-Precision	0.9964
Micro-Recall	0.8856
Exact Set Match Rate	0.6900

By ground–truth set size K				
K	n	Precision	Recall	Exact Set Match
2	300	1.0000	0.7717	0.5433
3	200	0.9925	0.8850	0.7350
4	300	0.9940	0.9633	0.9100
5	200	1.0000	0.8610	0.5350

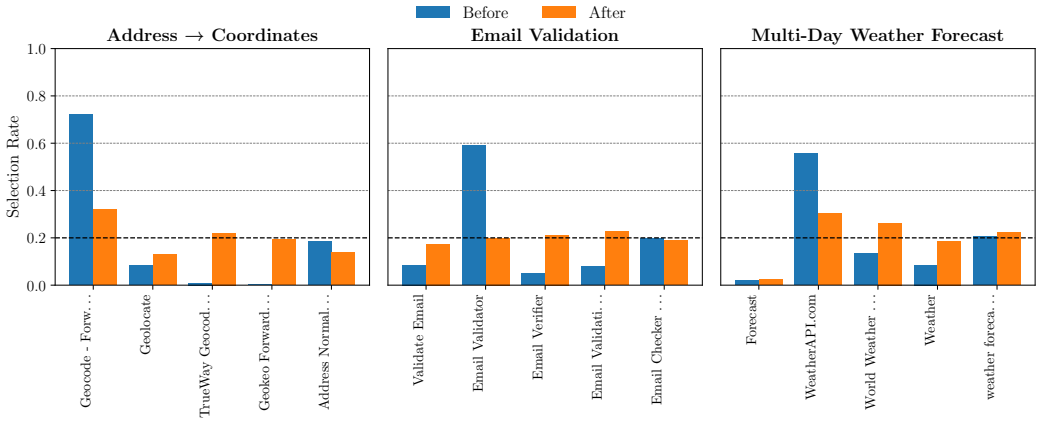

Figure 6: Selection distributions for ChatGPT 4.1-mini with and without utilizing our mitigation method across three clusters. Each subplot corresponds to one cluster, with the x-axis indicating the API in the cluster and the y-axis showing the fraction of times that API was chosen using the corresponding setup over 500 inference runs.

three clusters of functionally equivalent APIs. Where choices were far from uniform before, after the mitigation method is applied we notice an even spread of selection share. However, in the weather forecasting cluster (right), we see that one API is still being underutilized, perhaps being an indication that the model has a difficult time using this API correctly even when it is the only API available. The effectiveness of the mitigation method is further exemplified in Table 5, where a steep decrease in all our bias metrics after applying the mitigation method can be observed.

5 Conclusion

In this paper, we introduced the first benchmark for evaluating tool selection bias in LLMs. Our results establish tool selection bias as a real and potentially significant issue for tool-augmented (agentic) LLMs, with implications for user experience, operational cost, and marketplace fairness. This study offers a concrete starting point for understanding and mitigating this bias.

Limitations: This study is limited by using 100 synthetically generated user queries per cluster and a narrow seven-feature set for selection-rate predictions, limited model/repeat coverage, and a focus on APIs from RapidAPI and English queries. Although our choices were constrained by compute (our setup already required ∼500,000 inference runs), these factors may induce variance and restrict generality. **Future Work:** Future work should scale queries and clusters and enrich features with deeper semantic and structural signals to raise predictive power beyond the modest observed R^2. In addition, deploying more expressive models (e.g., boosted trees or deep nets) with cross-validation could capture higher-order interactions between tool features, further increasing their explanatory power. Lastly, broader replication across LLMs and runs would aid in quantifying variability.